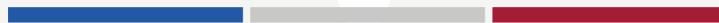




MIT SLOAN SCHOOL OF MANAGEMENT
MIT SCHWARZMAN COLLEGE OF COMPUTING



MODULE 3 UNIT 2
Media Set Video 3 Transcript

Module 3 Unit 2 Media Set Video 3 Transcript

Agentic AI capabilities

PAUL MCDONAGH-SMITH: So, Tim, I'm curious about the strategic calculus here. When you advise organizations through your MIT work, how do you help leaders think about what to build internally versus what to adopt as a platform? What capabilities around agentic AI are truly differentiating and worth investing in versus what's becoming, in your view, somewhat of a commodity infrastructure?

TIM KRASKA: I think this is really an excellent question because, like, you probably have seen these announcements around Claude Code, and it's right now everywhere in the media. It changes even my perspective over the last, like, one, two months, it changed drastically.

I think like the meaning of code is no longer the same in what you should buy and what you can build yourself. So now for AI very specifically, you still need like a frontier model. Like, right now, I would argue there are like three, four companies you can buy one of these frontier models from and give your workforce access to it. Like, it's a given. I think the tools which come after that, they are moving and changing very very quickly.

I would probably argue like something which enables everybody in your organization to experiment with building code by themselves. It's like one of the things. I also think like what software as a service means will change again going forward and there will be new tools coming out. So like at MIT we are working on something called G5, just like one of these examples on how we believe the organizations need to change going forward and what code even means going forward. Right? And that directly comes then back to what software should require because I don't think it will stay the same as what we have right now.

Emerging capabilities

MCDONAGH-SMITH: So, Tim, if our participants listening to our conversation here are building their strategic playbooks today for the near and next futures, what are the emerging capabilities or shifts that you think they should be designing for? And what current assumptions about agentic AI do you think will look slightly different in hindsight?

KRASKA: So if you think about it, like why do we buy software as a service? On one hand, you don't want to reimplement the code, right, like the program, whatever you use because somebody else did it, it's cheaper for you to just use it as a service. The other one is maintenance and running the service itself. Like, that's a nice reason of, like, a subscription model. You don't have to take care of any of that. Somebody else operates it for you. So now if you look at on how fast these models capabilities are evolving, I think with Claude Code, everybody suddenly realized that the cost of software is coming way down, and so you are questioning if you are still willing to pay these high premiums or just should build something by yourself.

The other one is still like the maintenance, and I think it's still not fully there, operating a service like still requires human nowadays, but I also think that we will get there eventually. Right? And so the whole meaning of software changes quite drastically. So the project we

currently have at MIT called G5 is like really looking into this and like the question we ask ourselves like almost two years ago was, “How would we design big complex systems, something like a database, an SAP system, CIM like HubSpot or Salesforce, if we know that most of the code will be written by AI?” Right? And how would we operate them as well later on? So the result of that is something which we call like a fifth-generation language. Right? And in the essence of it, what we are trying to do is make natural language in the sum structure which we call the system ontology, the new source code of the future.

So source code only becomes a derivative. And the new source code of the future is essentially the documents you write what the system is supposed to do, the intent. Yep. And everything else gets derived from that. So this is like we name it fifth-generation language because if you look at the evolution of languages, we started out with like machine code, binary code. Right? So punch cards, you put it into a machine and executed something. We very quickly realized this is horrible, and so we went to assembly languages very close to the hardware but in an abstraction way. But then assembly languages again, like, as close it is to the hardware, the harder the language is to use. So we invented higher-level languages like C++, Java, and so on, which these languages you no longer think about how the hardware functions, you no longer think about registers or, like AVX instructions. You think about objects and classes and functions. Right? Like a much higher level of obstruction.

In some restricted domains, we went for something which is called declarative languages where you specify what you want and then the system figures out how to achieve it. SQL is an example of that for data processing. The problem with declarative languages is only that we never were able to make it work in the broader scale like for general development. Now, with these new models, natural language for the first time could be the next evolution of programming languages where we make something which is partially underspecified, right, like the system description and then everything else gets automatically generated from it. If you find a bug, you no longer fix it in the code. You fix it actually in your documents, right, in the description.

The only downside right now is like you need to organize all that text in some form and so that's where we have invented this like system ontology. Right? So now if you put that further and you think further ahead what that means is ontologies or like semantic layers or some people nowadays would call it a world model, right? I'm just not a cool kid so therefore I cannot call it a world model.

That means just like if we are working down a semantic layer, right, and then software just becomes an artifact which is automatically derived from that. I think it changes fundamentally how companies will deploy software in the future because instead of maybe buying software, you buy best practices in form of ontologies, then you modify them for your business, and then you give it to the agents which automatically build the software, deploy it, and operate it going forward. This sounds like futuristic at the moment, but if you see how fast these things are developing, I don't think it's that far out anymore.

MCDONAGH-SMITH: Yeah. The future is getting closer, so to speak.

KRASKA: Much closer.

MCDONAGH-SMITH: How would you suggest to our business leaders that they almost create the conditions in their organizations to accept or be ready or be adaptive, perhaps is a better way of putting it? Because the companies I've just mentioned, one might expect that they have kind of a higher tolerance for kind of a higher adaptive kind of quotient. How would you advise our business leaders should be thinking about positioning kind of what we have here and what's coming to their employees, but also their customers?

KRASKA: I think, like, seeing it firsthand in some of these operations, what they do, I think it should be a bottom-up and top-down approach at the same time. So the bottom-up one, I think right now if you give your, like, particular in your IT department the power of these tools, they will start using it in certain domains and get value out of it and become more productive.

At the same time, I see or observe resistance of like going all the way and if you have been, let's say, a C++ developer for 30 years of your life, it's very hard to accept the reality that maybe in the future you no longer write code.

MCDONAGH-SMITH: Yeah. It's an identity shift.

KRASKA: It's an identity shift and some really jump on the train and immediately adapt and some others, they are like very very resistant and they come always up with some reasons not to do it. Some of them valid, some of them not, but like the future is in my opinion heading that way. So I would do it both ways—

MCDONAGH-SMITH: Like in parallel.

KRASKA: In parallel, like give them the tools and empower them to do just like little improvements, but at the same time, I'm pretty sure if you're on a large corporation right now, there are systems you have which are legacy. You always wanted to modernize them. You never got to it. It cost a fortune to operate them. Maybe try your approach like G5 what we are proposing as a tryout and just say like, "Okay, I take a radical new idea here. Let's see how far we can push it," and I think it will actually come way faster, these approaches, than we think.

MCDONAGH-SMITH: So thank you, Tim. We've arrived at the point in our fascinating conversation where I think I have to say thank you on behalf of all of our business leaders, all of the participants in our program here.

Very excited, kind of not just for the futures, but the way in which we can work with what we have in front of us here today. No time to waste, kind of get playing with things, kind of moving forward with our experimentation. A big thank you.

KRASKA: Thanks for having me.

SPEAKER: Did you understand all the concepts covered in this video? If you'd like to review any of the content, please click on the chapter menu.